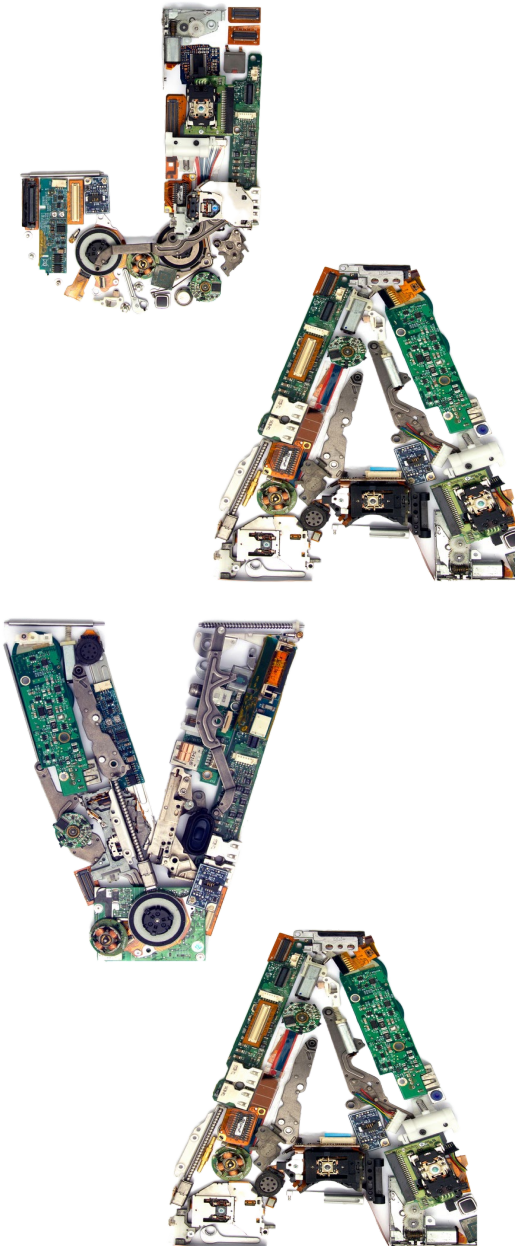


Programação Orientada a Objetos com Java e WEB

Banco de Dados – Exercício



Exercício de programação

Criar as tabelas a seguir no banco de dados Oracle

java_vendedor	
Campo	Tipo
ID_vendedor (pk)	number
nome_vendedor	varchar2(50)

java_venda	
Campo	Tipo
ID_venda (pk)	number
ID_vendedor (fk)	number
total	number(10,2)
data	date

Definição de chave estrangeira

Para definir uma **chave estrangeira** em uma tabela, você precisa criar uma relação entre uma coluna (ou conjunto de colunas) da tabela filha e a coluna correspondente na tabela pai. A **chave estrangeira** impõe uma restrição de integridade referencial, garantindo que os valores na coluna da tabela filha correspondam aos valores existentes na coluna da tabela pai.

Para adicionar uma chave estrangeira usando a instrução **ALTER TABLE**.

```
ALTER TABLE <tabela_filha>
ADD CONSTRAINT <nome_da_chave_estrangeira>
FOREIGN KEY (<coluna_filha>)
REFERENCES <tabela_pai> (<coluna_pai>);
```

Definição de chave estrangeira

ON DELETE CASCADE: Se você quiser que, ao deletar um registro na tabela pai, os registros correspondentes na tabela filha também sejam deletados automaticamente, adicione **ON DELETE CASCADE** na definição da chave estrangeira:

```
ALTER TABLE employees
ADD CONSTRAINT fk_department
FOREIGN KEY (department_id)
REFERENCES departments(department_id)
ON DELETE CASCADE;
```

Definição de chave estrangeira

ON DELETE SET NULL: Se quiser que os registros na tabela filha tenham suas chaves estrangeiras definidas como **NULL** ao deletar o registro na tabela pai:

```
ALTER TABLE employees
ADD CONSTRAINT fk_department
FOREIGN KEY (department_id)
REFERENCES departments(department_id)
ON DELETE SET NULL;
```

Inserção de datas no banco de dados Oracle

Para inserir valores de data diretamente em uma tabela usando o **Oracle SQL Developer**, você pode utilizar a função **TO_DATE** para formatar a data de forma explícita ou inserir a data em um formato padrão aceito pelo Oracle. Exemplos:

```
INSERT INTO tabela_exemplo (coluna_data)
VALUES (TO_DATE('25/06/2024', 'DD/MM/YYYY'));
```

```
INSERT INTO teste_venda
VALUES (2, 'Patrícia', 1500.25, TO_DATE('10/08/2024'));
```

```
INSERT INTO tabela_exemplo (coluna_data)
VALUES (TO_DATE('10/08/2024 14:30:00', 'DD/MM/YYYY HH24:MI:SS'));
```

```
INSERT INTO tabela_exemplo (coluna_data)
VALUES (SYSDATE);
```

Instruções SQL

Considere as seguintes instruções SQL na tabela **users** contendo os campos: **ID** (number), **nome** (varchar2(100)) e **CPF** (number).

```
insert into users values (1, 'Pedro Almeida', 12345678996);
```

```
select * from users;
```

```
select id, nome from users where cpf = 12345678996;
```

```
update users set nome = 'Pedro Antonio Almeida', cpf = 123  
where id = 1;
```

```
delete from users where cpf = 123;
```


Manipulação de datas no Java

Em um programa Java, a classe **LocalDate** da API **java.time** é amplamente utilizada para manipular datas de forma intuitiva e segura. Ela oferece uma abordagem moderna para representar apenas a data, sem informações de tempo, sendo ideal para operações como cálculos de datas e formatação.

Ao interagir com bancos de dados, é necessário converter o objeto **LocalDate** em uma instância de **java.sql.Date**, pois esta última é a classe compatível com o formato de data exigido pela maioria dos bancos de dados relacionais.

A conversão é simples e direta, garantindo que a data seja armazenada corretamente no banco de dados.

Manipulação de datas no Java

1

define o formato da data que será manipulada em um programa Java.
java.time.format.DateTimeFormatter

```
String dataString = "10/08/2025";
```

```
1 DateTimeFormatter formato = DateTimeFormatter.ofPattern("dd/MM/yyyy");
```

2

```
2 LocalDate localDate = LocalDate.parse(dataString, formato);
```

converte a data string para **LocalDate** aplicando o formato especificado

```
3 Date data = Date.valueOf(localDate);
```

3

converte **LocalDate** para **Sql Date** para inserir no banco de dados

```
4 LocalDate dataAux = data.toLocalDate();
```

```
String dataImpressao = dataAux.format(formato);
```

4

Converte **Sql Date** (data recuperada do banco de dados) em **LocalDate**

```
System.out.println(dataImpressao);
```

Exercício de programação

Desenvolver uma aplicação em Java que permite ao usuário da aplicação realizar as seguintes operações nas tabelas **java_vendedor** e **java_venda**:

1. Cadastrar vendedores.
2. Cadastrar vendas.
3. Listar vendas (nome do vendedor e data da venda).
4. Listar vendas (nome do vendedor e total das vendas).
5. Alterar nome de um vendedor específico.
6. Remover uma venda específica.
7. Remover um vendedor específico.